Cairo University, Faculty of Engineering

Department of Electronics and Electrical Communication.

Exercise: Full-Stack AXI Integration of a Custom Accelerator in PULP

Project Title:

Hardware-Accelerated Implementation for Massive MIMO Detection in 5G/6G Systems

Sponsored by:

Analog Devices Inc. (ADI), Egypt

Under supervision of:

Prof. Ahmed Hesham

Table of Contents

Full-Stack AXI Integration Problem Statement

In this exercise, the objective is to integrate a custom hardware accelerator into the PULPissimo system using a full-stack approach that spans hardware description, register-file generation, AXI interconnect integration, and software driver development.

The accelerator to be integrated is a dummy arithmetic unit (`wide_alu`) that operates on wide data types. It accepts two 256-bit operands, performs a user-selectable arithmetic operation (e.g., addition or multiplication), and produces a 512-bit result. In addition, the accelerator supports a programmable delay parameter (referred to as a deacceleration factor), which controls the number of cycles before the result becomes available.

The `wide_alu` module exposes a non-standard interface and therefore cannot be directly accessed by the RISC-V core using memory-mapped I/O. To make the accelerator programmable via standard load/store instructions, it must be wrapped with a register file and connected to the system AXI interconnect.

Scope of the Exercise

The exercise covers the complete integration flow of a custom IP into PULPissimo, including:

- Generation of a memory-mapped register file using **reggen** from a provided HJSON register description
- Wrapping of the accelerator with a PULP-compatible AXI slave interface
- Conversion between AXI transactions and the Generic Register Interface protocol
- Integration of the wrapped IP into the PULPissimo AXI crossbar
- Extension of the SoC address map to include the new peripheral
- Integration of a software driver into `pulp-runtime`
- Validation of the complete hardware/software stack using a test application

Hardware Integration Requirements

The hardware integration consists of the following tasks:

- Analyze the provided `wide_alu` module to understand its interface and functionality
- Generate a register file from `wide_alu.hjson` using the reggen tool
- Create a SystemVerilog wrapper (`wide_alu_top`) that:
    - Exposes a single AXI slave interface
    - Converts AXI transactions to the Generic Register Interface
    - Connects the generated register file to the `wide_alu` module
- Integrate the wrapped accelerator into `pulp_soc` by:
    - Instantiating the AXI interface
    - Connecting it to the AXI crossbar
    - Adding a new memory-mapped address region for the accelerator
    - Updating AXI address decoding rules accordingly

Software Integration Requirements

The software integration consists of the following tasks:

- Integrate the provided partial driver implementation into `pulp-runtime`
- Generate and integrate the C header file from `wide_alu.hjson`
- Complete the driver implementation to:
  - Configure the accelerator operation
  - Trigger computations
  - Read back the computation result
- Validate the driver by running a provided test application (`main.c`) on PULPissimo

Success Criteria

The exercise is considered successfully completed when:

- The accelerator is accessible as a memory-mapped AXI peripheral
- The PULPissimo build completes without errors
- The software driver communicates with the accelerator using generated register definitions
- The provided test application executes correctly and prints the expected result (`15`) to standard output

Hardware Integration Steps

Step 0 – initial setup for the Wide-ALU IP

Commands:

```
cd ~/GP

mkdir  rtl/wide_alu

cd rtl/wide_alu

touch wide_alu.hjson

cd ~/GP
```

Step 1 – Locating and Preparing the Reggen Tool

Before generating any register files, the **reggen (regtool)** utility must be located and verified. This tool is responsible for converting the `wide_alu.hjson` register description into:

- SystemVerilog RTL
- C header files
- Documentation

For PULP-based systems, **reggen must come from the `register_interface` IP**, as it is patched to support the *Generic Register Interface Protocol* used in this exercise.

Locating reggen

Path:

```
GP/.bender/git/checkouts/register_interface
1e5f22ae6b2b5b5d/vendor/lowrisc_opentitan/util/regtool.py
```

This version is compatible with:

- PULP register interface
- Generic register bus (REG_BUS)
- AXI-to-register protocol converters used later

Exporting the reggen path

Set an environment variable pointing to the reggen tool:

```
export REGGEN=$HOME/GP/.bender/git/checkouts/register_interface-
1e5f22ae6b2b5b5d/vendor/lowrisc_opentitan/util/regtool.py
```

Verify that the tool is accessible:

```
python3 $REGGEN --help
```

Resolving dependencies (hjson)

The reggen tool depends on the Python package **hjson**.
If the tool fails with:

```
ModuleNotFoundError: No module named 'hjson'
```

Install the dependency locally:

```
pip3 install --user hjson
```

Then Re-run the help command to confirm successful setup:

```
python3 $REGGEN --help
```

Step Summary

- The correct **reggen tool** was identified from the `register_interface` IP
- Required Python dependencies were installed
- reggen execution was verified successfully
- The environment is now ready for **register file generation**
- At this point, **no files have been generated yet**. This step only ensures that the tooling required for reggen-based generation is correctly set up.

Step 2 – Attempting Register File Generation with reggen

Attempt to generate the SystemVerilog register file for the wide_alu IP using the reggen tool and identify any tooling or compatibility issues.

Action

Attempt to generate SystemVerilog RTL:

```
python3 $REGGEN -r rtl/wide_alu/wide_alu.hjson --outdir rtl/wide_alu/
```

Result

The reggen tool fails with the following error:

```
ERROR: block at wide_alu.hjson doesn't have the right keys.
The following required fields were missing: bus_interfaces.
The following unexpected fields were found: bus_device, bus_host.
```

Retrying with validation disabled yields the same result:

```
python3 $REGGEN -r rtl/wide_alu/wide_alu.hjson --outdir rtl/wide_alu/
--novalidate
```

Analysis

- The `wide_alu.hjson` register description uses an **older / patched reggen schema** (`bus_device`, `bus_host`).
- The locally available `regtool.py` expects a **newer OpenTitan reggen schema** (`bus_interfaces`).
- No schema-compatible reggen version exists in the current PULP workspace.
- A full search of the PULP source tree confirms that:
    - Existing PULP IPs do **not** use reggen-generated register files.

- `wide_alu` is the **only IP** with a register-style `.hjson`, intended as a **training example**.

Conclusion

The failure is **not caused by an error in `wide_alu.hjson`**, but by a **tool–schema mismatch** between:

- the register description provided by the exercise, and
- the reggen versions available locally.

So,

- We will treat `wide_alu.hjson` as a **reference register specification**.
- Use the **provided / assumed generated register RTL** as intended by the tutorial.
- Continue the exercise by manually integrating the wrapper, AXI interface, and driver.

Step Summary

An attempt was made to generate the `wide_alu` register file using reggen. The process failed due to a schema incompatibility between the provided `.hjson` and available reggen tools. This behavior is expected for this training exercise. As a result, register generation is skipped, and the exercise proceeds using the provided register specification.

Step 2 Register Description Fix – Making `wide_alu.hjson` Compatible with PULP reggen

At this point, we attempted to run `regtool.py` (reggen) on the provided `wide_alu.hjson` in order to generate the SystemVerilog register file.

However, the initial attempt failed with the following error:

```
ERROR: block at wide_alu.hjson doesn't have the right keys.
The following required fields were missing: bus_interfaces.
The following unexpected fields were found: bus_device, bus_host.
```

This indicates a **schema mismatch** between:

- the register description format used in the tutorial, and
- the reggen version vendored inside the current PULP / register_interface repository.

Root Cause

The tutorial's original `wide_alu.hjson` was written for an **older reggen schema**, which used:

```
bus_device: "reg"
bus_host: ""
```

The **current reggen** used in PULP expects the newer OpenTitan-style schema, which requires:

```
bus_interfaces: [
  { protocol: "reg_iface", direction: "device" }
]
```

Because of this:

- reggen refused to validate the file
- `--novalidate` does **not** bypass schema validation
- generation was impossible without adapting the file

Decision

Instead of trying to patch reggen or downgrade tooling, we **updated the register description** to match the current reggen expectations.

This is safe and correct because:

- The register semantics did not change
- Only the **metadata format** was updated
- The generated RTL matches the intended behavior of the IP
- This is exactly how OpenTitan-style reggen is meant to be used today

Updated `wide_alu.hjson`

The following changes were applied:

1. Replaced legacy bus fields

Removed

```
bus_device: "reg",
bus_host: "",
```

Added

```
bus_interfaces: [
    { protocol: "reg_iface", direction: "device" }
],
```

2. Explicitly enabled hardware write-enable (`qe`) signals

For control registers that rely on edge-triggered semantics (`.q & .qe` in hardware), we explicitly added:

`hwqe: "true"`

This was done for:

- `CTRL1` (TRIGGER, CLEAR_ERR)
- `CTRL2` (OPSEL, DELAY)

This ensures reggen generates:

- `.q` (stored value)
- `.qe` (write-enable pulse)

which matches how the wrapper drives the accelerator.

Final `wide_alu.hjson` (Validated and Working)

After this modification:

```
{
  "name": "wide_alu",
  "clock_primary": "clk_i",
  "reset_primary": "rst_ni",
  "bus_interfaces": [
    { "protocol": "reg_iface", "direction": "device" }
  ],
  "regwidth": 32,
  "registers": [
    {
      "multireg": {
        "name": "OP_A",
        "desc": "Subword of Operand A.",
        "count": "8",
        "cname": "OP_A",
        "swaccess": "wo",
```

```
    "fields": [
      { "bits": "31:0", "desc": "Operand A data" }
    ]
  }
},
{
  "multireg": {
    "name": "OP_B",
    "desc": "Subword of Operand B.",
    "count": "8",
    "cname": "OP_B",
    "swaccess": "wo",
    "fields": [
      { "bits": "31:0", "desc": "Operand B data" }
    ]
  }
},
{
  "multireg": {
    "name": "RESULT",
    "desc": "Subword of results.",
    "count": "16",
    "cname": "RESULT",
    "swaccess": "ro",
    "hwaccess": "hwo",
    "hwext": "true",
```

```
    "fields": [
      { "bits": "31:0", "desc": "Result data" }
    ]
  }
},
{
  "name": "CTRL1",
  "desc": "Controls clear and trigger signal of the deaccelerator.",
  "swaccess": "wo",
  "hwaccess": "hro",
  "hwext": "true",
  "hwqe": "true",
  "fields": [
    { "bits": "0", "name": "TRIGGER", "desc": "Trigger the operation" },
    { "bits": "1", "name": "CLEAR_ERR", "desc": "Clear error flags" }
  ]
},
{
  "name": "CTRL2",
  "desc": "Configures the operation and its delay within the deaccelerator.",
  "swaccess": "rw",
  "hwaccess": "hrw",
  "hwext": "true",
  "hwqe": "true",
  "fields": [
    { "bits": "2:0", "name": "OPSEL", "desc": "Operation selection configuration" },
```

```
      { "bits": "23:16", "name": "DELAY", "desc": "Deacceleration delay cycles" }
    ]
   },
   {
    "name": "STATUS",
    "desc": "Contains the current status of the Deaccelerator.",
    "swaccess": "ro",
    "hwaccess": "hwo",
    "hwext": "true",
    "fields": [
     { "bits": "1:0", "name": "CODE", "desc": "Current status code" }
    ]
   }
  ]
}
```

- `regtool.py -r wide_alu.hjson` runs successfully
- SystemVerilog register files are generated correctly
- Generated structs match the wrapper wiring (`.q`, `.qe`, `.d`)
- No behavior deviation from the tutorial intent

Step 3 – Creating the wide_alu_top Wrapper Skeleton

This step introduces the **top-level wrapper module** for the `wide_alu` accelerator.
At this stage, the goal is **only to define the module interface and structure**, without any internal logic or protocol conversion yet.

Step 3.1 – Create the Wrapper Skeleton

File to create

```
$HOME/GP/rtl/wide_alu/wide_alu_top.sv
```

Purpose

- Define a PULP-compatible top-level wrapper
- Expose a single AXI slave interface
- Provide clock, reset, and test-mode signals
- Prepare the module for later integration steps

At this point:

- No protocol converters

- No register file
- No accelerator instantiation
  Only the **module shell**.

Code added in this step

```
module wide_alu_top #(
    parameter int unsigned AXI_ADDR_WIDTH = 32,
    localparam int unsigned AXI_DATA_WIDTH = 32,
    parameter int unsigned AXI_ID_WIDTH   = -1,
    parameter int unsigned AXI_USER_WIDTH = -1
) (
    input  logic    clk_i,
    input  logic    rst_ni,
    input  logic    test_mode_i,

    AXI_BUS.Slave   axi_slave
);

endmodule
```

Notes

- `AXI_BUS.Slave` is used to expose the accelerator as an AXI peripheral
- Address, data, ID, and user widths are parameterized to match `pulp_soc`
- No internal logic is present yet — this is intentional

Step Summary

In this step, we created the initial `wide_alu_top` wrapper module.
The module defines the required clock, reset, and test-mode signals and exposes a single AXI slave interface. No internal logic or protocol conversion is implemented yet; this file serves as the structural foundation for integrating the wide ALU accelerator into `pulp_soc`.

Step 3.2 – Adding the Register Interface Includes

In this step, we prepare the wrapper to work with the **Generic Register Interface** used by `reggen` and the AXI-to-register protocol converter.

Specifically, we include the required SystemVerilog macro definitions that:

- Define register bus request/response structures
- Provide assignment macros between interfaces and structs

No functional logic is added yet.

File to Modify

`$HOME/GP/rtl/wide_alu/wide_alu_top.sv`

Code Changes in This Step

Add the following include directives **at the top of the file**, before the module declaration:

```
`include "register_interface/typedef.svh"
`include "register_interface/assign.svh"
```

Current State of `wide_alu_top.sv`

```
`include "register_interface/typedef.svh"
`include "register_interface/assign.svh"

module wide_alu_top #(
    parameter int unsigned AXI_ADDR_WIDTH = 32,
    localparam int unsigned AXI_DATA_WIDTH = 32,
    parameter int unsigned AXI_ID_WIDTH   = -1,
    parameter int unsigned AXI_USER_WIDTH = -1
) (
    input  logic    clk_i,
    input  logic    rst_ni,
    input  logic    test_mode_i,

    AXI_BUS.Slave   axi_slave
);

endmodule
```

Notes

- These includes are required later for:
  - Converting `REG_BUS` interfaces into structs
  - Connecting the AXI protocol converter to the autogenerated register file
- No behavior or connectivity changes occur at this stage

Step Summary

In this step, we added the required `register_interface` include files to the `wide_alu_top` wrapper. These includes provide the macro definitions needed for register bus typedefs and assignments, which will be used in subsequent steps when integrating the AXI-to-register protocol converter and the autogenerated register file.

Step 3.3 – Importing the reggen-generated Register Package

In this step, we make the wrapper aware of the **types generated by reggen**.
The autogenerated register package defines structured connections between:

- the register file and
- the hardware accelerator logic

These types will be used later to connect the register file to the `wide_alu` IP.

At this stage:

- No instances are created
- No logic is connected
  We only **import the types**.

File to Modify

```
$HOME/GP/rtl/wide_alu/wide_alu_top.sv
```

Code Changes in This Step

Add the following imports **inside the module**, immediately after the port list:

```
    import wide_alu_reg_pkg::wide_alu_reg2hw_t;
    import wide_alu_reg_pkg::wide_alu_hw2reg_t;
```

Current State of `wide_alu_top.sv`

```
`include "register_interface/typedef.svh"
`include "register_interface/assign.svh"

module wide_alu_top #(
    parameter int unsigned AXI_ADDR_WIDTH = 32,
    localparam int unsigned AXI_DATA_WIDTH = 32,
    parameter int unsigned AXI_ID_WIDTH   = -1,
    parameter int unsigned AXI_USER_WIDTH = -1
) (
    input  logic    clk_i,
    input  logic    rst_ni,
    input  logic    test_mode_i,

    AXI_BUS.Slave   axi_slave
);

    import wide_alu_reg_pkg::wide_alu_reg2hw_t;
    import wide_alu_reg_pkg::wide_alu_hw2reg_t;

endmodule
```

Notes

- `wide_alu_reg2hw_t`
  Carries configuration and control signals **from software → hardware**
- `wide_alu_hw2reg_t`
  Carries status and result signals **from hardware → software**
- These types are autogenerated from `wide_alu.hjson`

Step Summary

In this step, we imported the register-to-hardware and hardware-to-register struct types generated by reggen. These imports allow the wrapper to later declare wiring signals that connect the autogenerated register file to the wide ALU accelerator.

Step 3.4 – Declaring the Internal `REG_BUS` Interface

In this step, we declare an **internal register bus interface** that will sit between:

- the AXI-to-register protocol converter, and
- the autogenerated register file.

This interface represents the **Generic Register Interface Protocol** used throughout PULP for memory-mapped peripherals.

At this stage:

- No AXI conversion yet
- No register file instance yet
  We only **declare the interface**.

File to Modify

```
$HOME/GP/rtl/wide_alu/wide_alu_top.sv
```

Code Changes in This Step

Add the following declaration **inside the module body**, after the `import` statements:

```
    // Declare the internal Register Bus interface
    REG_BUS #(
        .ADDR_WIDTH(32),
        .DATA_WIDTH(32)
    ) axi_to_regfile();
```

Current State of `wide_alu_top.sv`

```
`include "register_interface/typedef.svh"
`include "register_interface/assign.svh"

module wide_alu_top #(
    parameter int unsigned AXI_ADDR_WIDTH = 32,
    localparam int unsigned AXI_DATA_WIDTH = 32,
    parameter int unsigned AXI_ID_WIDTH   = -1,
    parameter int unsigned AXI_USER_WIDTH = -1
) (
    input  logic    clk_i,
    input  logic    rst_ni,
```

```
    input  logic    test_mode_i,

    AXI_BUS.Slave   axi_slave
);

    import wide_alu_reg_pkg::wide_alu_reg2hw_t;
    import wide_alu_reg_pkg::wide_alu_hw2reg_t;

    // Declare the internal Register Bus interface
    REG_BUS #(
        .ADDR_WIDTH(32),
        .DATA_WIDTH(32)
    ) axi_to_regfile();

endmodule
```

Notes

- `REG_BUS` is a SystemVerilog interface defined in the `register_interface` IP
- It abstracts a simple register read/write protocol
- Using an interface here simplifies later connections and conversions

Step Summary

In this step, we declared an internal `REG_BUS` interface that will be used to carry register transactions between the AXI-to-register protocol converter and the autogenerated register file. This interface forms the backbone of the control path between software and the wide ALU accelerator.

Step 3.5 – Declaring Register-to-Hardware and Hardware-to-Register Wiring Structs

In this step, we declare the **wiring signals** that will later connect:

- the autogenerated register file
- to the actual `wide_alu` hardware accelerator.

These signals are **structs**, generated by `reggen`, and they provide a typed, field-level connection between software-visible registers and hardware logic.

At this stage:

- No register file instance yet
- No accelerator instance yet
  We only declare the signals.

File to Modify

`$HOME/GP/rtl/wide_alu/wide_alu_top.sv`

Code Changes in This Step

Add the following declarations **inside the module body**, after the `REG_BUS` interface declaration:

```
    // Wiring signals between register file and wide_alu IP
    wide_alu_reg2hw_t reg_file_to_ip;
    wide_alu_hw2reg_t ip_to_reg_file;
```

Current State of `wide_alu_top.sv`

```
`include "register_interface/typedef.svh"
`include "register_interface/assign.svh"

module wide_alu_top #(
    parameter int unsigned AXI_ADDR_WIDTH = 32,
    localparam int unsigned AXI_DATA_WIDTH = 32,
    parameter int unsigned AXI_ID_WIDTH   = -1,
    parameter int unsigned AXI_USER_WIDTH = -1
) (
    input  logic    clk_i,
    input  logic    rst_ni,
    input  logic    test_mode_i,

    AXI_BUS.Slave   axi_slave
);

    import wide_alu_reg_pkg::wide_alu_reg2hw_t;
    import wide_alu_reg_pkg::wide_alu_hw2reg_t;

    // Declare the internal Register Bus interface
    REG_BUS #(
        .ADDR_WIDTH(32),
        .DATA_WIDTH(32)
    ) axi_to_regfile();

    // Wiring signals between register file and wide_alu IP
    wide_alu_reg2hw_t reg_file_to_ip;
    wide_alu_hw2reg_t ip_to_reg_file;

endmodule
```

Notes

- `reg_file_to_ip`
  Carries control, configuration, and operand data **from registers to hardware**
- `ip_to_reg_file`
  Carries result and status data **from hardware back to registers**
- These structs mirror the structure defined in `wide_alu.hjson`

Step Summary

In this step, we declared the register-to-hardware and hardware-to-register wiring structs generated by reggen. These signals will later be used to connect the autogenerated register file to the wide ALU accelerator in a structured and type-safe way.

Step 3.6 – Instantiating the AXI-to-Register Protocol Converter

In this step, we instantiate the **AXI-to-Register protocol converter** (`axi_to_reg_intf`).

This block:

- Accepts AXI transactions from the SoC AXI crossbar
- Converts them into transactions on the **Generic Register Interface**
- Acts as the bridge between AXI and the autogenerated register file

At the end of this step:

- AXI requests can reach the internal register bus
- The register file is **not yet connected**

File to Modify

`$HOME/GP/rtl/wide_alu/wide_alu_top.sv`

Code Changes in This Step

Add the following instantiation **inside the module body**, after the `REG_BUS` declaration and wiring struct declarations:

```
    // Instantiate the AXI to Register Interface converter
    axi_to_reg_intf #(
        .ADDR_WIDTH ( AXI_ADDR_WIDTH ),
        .DATA_WIDTH ( AXI_DATA_WIDTH ),
        .ID_WIDTH   ( AXI_ID_WIDTH   ),
        .USER_WIDTH ( AXI_USER_WIDTH ),
        .DECOUPLE_W ( 0              )
    ) i_axi2reg (
        .clk_i,
        .rst_ni,
        .testmode_i ( test_mode_i ),
        .in         ( axi_slave  ),
        .reg_o      ( axi_to_regfile )
    );
```

Current State of `wide_alu_top.sv`

```
`include "register_interface/typedef.svh"
`include "register_interface/assign.svh"

module wide_alu_top #(
    parameter int unsigned AXI_ADDR_WIDTH = 32,
```

```
    localparam int unsigned AXI_DATA_WIDTH = 32,
    parameter int unsigned AXI_ID_WIDTH   = -1,
    parameter int unsigned AXI_USER_WIDTH = -1
) (
    input  logic    clk_i,
    input  logic    rst_ni,
    input  logic    test_mode_i,

    AXI_BUS.Slave   axi_slave
);

    import wide_alu_reg_pkg::wide_alu_reg2hw_t;
    import wide_alu_reg_pkg::wide_alu_hw2reg_t;

    // Internal Register Bus interface
    REG_BUS #(
        .ADDR_WIDTH(32),
        .DATA_WIDTH(32)
    ) axi_to_regfile();

    // Wiring signals between register file and wide_alu IP
    wide_alu_reg2hw_t reg_file_to_ip;
    wide_alu_hw2reg_t ip_to_reg_file;

    // AXI to Register Interface protocol converter
    axi_to_reg_intf #(
        .ADDR_WIDTH ( AXI_ADDR_WIDTH ),
        .DATA_WIDTH ( AXI_DATA_WIDTH ),
        .ID_WIDTH   ( AXI_ID_WIDTH   ),
        .USER_WIDTH ( AXI_USER_WIDTH ),
        .DECOUPLE_W ( 0              )
    ) i_axi2reg (
        .clk_i,
        .rst_ni,
        .testmode_i ( test_mode_i ),
        .in         ( axi_slave  ),
        .reg_o      ( axi_to_regfile )
    );

endmodule
```

Notes

- `axi_slave` is the **AXI-facing port** of the wrapper
- `axi_to_regfile` is the **internal REG_BUS interface**
- No register file or accelerator is connected yet
- This matches exactly the structure used in the tutorial

Step Summary

In this step, we instantiated the AXI-to-Register protocol converter. This component bridges the AXI slave interface of the wide ALU wrapper to the internal generic register bus, enabling AXI-based register accesses in later steps.

Step 3.7 – Converting the REG_BUS Interface to reggen Structs

Adapt the `REG_BUS` SystemVerilog interface produced by the AXI-to-register protocol converter to the struct-based interface expected by the reggen-generated register file.

Background

The reggen-generated register file (`wide_alu_reg_top`) does not use SystemVerilog interfaces. Instead, it communicates using plain request/response structs that implement the *Generic Register Interface Protocol*.

Since the AXI-to-register protocol converter outputs a `REG_BUS` interface, an explicit conversion layer is required.

Modification Details

1. Declare register request and response struct types

```
`REG_BUS_TYPEDEF_REQ(reg_req_t, addr_t, data_t, strb_t)
`REG_BUS_TYPEDEF_RSP(reg_rsp_t, data_t)
```

These macros define packed struct types that exactly match the Generic Register Interface request and response formats.

2. Declare wiring signals for the structs

```
reg_req_t to_reg_file_req;
reg_rsp_t from_reg_file_rsp;
```

These signals act as the logical connection between the protocol converter and the autogenerated register file.

3. Convert between interface and structs

```
`REG_BUS_ASSIGN_TO_REQ(to_reg_file_req, axi_to_regfile)
`REG_BUS_ASSIGN_FROM_RSP(axi_to_regfile, from_reg_file_rsp)
```

These macros generate continuous assignments that:

- Map signals from the `REG_BUS` interface into the request struct
- Map signals from the response struct back into the `REG_BUS` interface

No additional logic or latency is introduced; this is a pure signal-level adaptation.

Code Changes in This Step

Add the following code **after the AXI-to-register converter instantiation**:

```
    // Convert the REG_BUS interface to the struct signals used by the
autogenerated register file
```

```
    typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
    typedef logic [AXI_DATA_WIDTH-1:0] data_t;
    typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;

    `REG_BUS_TYPEDEF_REQ(reg_req_t, addr_t, data_t, strb_t)
    `REG_BUS_TYPEDEF_RSP(reg_rsp_t, data_t)

    reg_req_t to_reg_file_req;
    reg_rsp_t from_reg_file_rsp;

    `REG_BUS_ASSIGN_TO_REQ(to_reg_file_req, axi_to_regfile)
    `REG_BUS_ASSIGN_FROM_RSP(axi_to_regfile, from_reg_file_rsp)
```

Current State of `wide_alu_top.sv`

```
`include "register_interface/typedef.svh"
`include "register_interface/assign.svh"

module wide_alu_top #(
    parameter int unsigned AXI_ADDR_WIDTH = 32,
    localparam int unsigned AXI_DATA_WIDTH = 32,
    parameter int unsigned AXI_ID_WIDTH   = -1,
    parameter int unsigned AXI_USER_WIDTH = -1
) (
    input  logic    clk_i,
    input  logic    rst_ni,
    input  logic    test_mode_i,

    AXI_BUS.Slave   axi_slave
);

    import wide_alu_reg_pkg::wide_alu_reg2hw_t;
    import wide_alu_reg_pkg::wide_alu_hw2reg_t;

    // Internal Register Bus interface
    REG_BUS #(
        .ADDR_WIDTH(32),
        .DATA_WIDTH(32)
    ) axi_to_regfile();

    // Wiring signals between register file and wide_alu IP
    wide_alu_reg2hw_t reg_file_to_ip;
    wide_alu_hw2reg_t ip_to_reg_file;

    // AXI to Register Interface protocol converter
    axi_to_reg_intf #(
        .ADDR_WIDTH ( AXI_ADDR_WIDTH ),
        .DATA_WIDTH ( AXI_DATA_WIDTH ),
        .ID_WIDTH   ( AXI_ID_WIDTH   ),
        .USER_WIDTH ( AXI_USER_WIDTH ),
        .DECOUPLE_W ( 0              )
    ) i_axi2reg (
        .clk_i,
        .rst_ni,
        .testmode_i ( test_mode_i ),
        .in         ( axi_slave  ),
```

```
        .reg_o      ( axi_to_regfile )
    );

    // REG_BUS interface to struct conversion
    typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
    typedef logic [AXI_DATA_WIDTH-1:0] data_t;
    typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;

    `REG_BUS_TYPEDEF_REQ(reg_req_t, addr_t, data_t, strb_t)
    `REG_BUS_TYPEDEF_RSP(reg_rsp_t, data_t)

    reg_req_t to_reg_file_req;
    reg_rsp_t from_reg_file_rsp;

    `REG_BUS_ASSIGN_TO_REQ(to_reg_file_req, axi_to_regfile)
    `REG_BUS_ASSIGN_FROM_RSP(axi_to_regfile, from_reg_file_rsp)

endmodule
```

Notes

- `to_reg_file_req` connects **AXI → register file**
- `from_reg_file_rsp` connects **register file → AXI**
- The macros guarantee correct field mapping and widths
- This approach avoids manual signal wiring errors

Step Summary

- Defined request and response struct types for the Generic Register Interface
- Declared struct-based wiring signals
- Converted between the `REG_BUS` interface and reggen-compatible structs
- Enabled seamless integration between the AXI protocol converter and the autogenerated register file

Step 3.8 – Instantiating the Autogenerated Register File

In this step, we instantiate the **reggen-generated register file** and connect it to:

- the AXI-to-register protocol converter (via struct signals), and
- the internal wiring structs that will later connect to the `wide_alu` hardware.

At this point:

- The AXI side is already converted into `reg_req_t` / `reg_rsp_t`
- The register file becomes the central control point
- The accelerator is **not yet connected**

File to Modify

`$HOME/GP/rtl/wide_alu/wide_alu_top.sv`

Code Changes in This Step

Add the following instantiation **after the REG_BUS-to-struct conversion**:

```
    // Instance of the auto-generated Register File
    wide_alu_reg_top #(
        .reg_req_t(reg_req_t),
        .reg_rsp_t(reg_rsp_t)
    ) i_regfile (
        .clk_i,
        .rst_ni,
        .devmode_i(1'b1),

        // From AXI-to-register protocol converter
        .reg_req_i(to_reg_file_req),
        .reg_rsp_o(from_reg_file_rsp),

        // Connections to the wide_alu IP
        .reg2hw(reg_file_to_ip),
        .hw2reg(ip_to_reg_file)
    );
```

Current State of `wide_alu_top.sv`

```
`include "register_interface/typedef.svh"
`include "register_interface/assign.svh"

module wide_alu_top #(
    parameter int unsigned AXI_ADDR_WIDTH = 32,
    localparam int unsigned AXI_DATA_WIDTH = 32,
    parameter int unsigned AXI_ID_WIDTH   = -1,
    parameter int unsigned AXI_USER_WIDTH = -1
) (
    input  logic    clk_i,
    input  logic    rst_ni,
    input  logic    test_mode_i,

    AXI_BUS.Slave   axi_slave
);

    import wide_alu_reg_pkg::wide_alu_reg2hw_t;
    import wide_alu_reg_pkg::wide_alu_hw2reg_t;

    // Internal Register Bus interface
    REG_BUS #(
        .ADDR_WIDTH(32),
        .DATA_WIDTH(32)
    ) axi_to_regfile();

    // Wiring signals between register file and wide_alu IP
    wide_alu_reg2hw_t reg_file_to_ip;
    wide_alu_hw2reg_t ip_to_reg_file;
```

```
    // AXI to Register Interface protocol converter
    axi_to_reg_intf #(
        .ADDR_WIDTH ( AXI_ADDR_WIDTH ),
        .DATA_WIDTH ( AXI_DATA_WIDTH ),
        .ID_WIDTH   ( AXI_ID_WIDTH   ),
        .USER_WIDTH ( AXI_USER_WIDTH ),
        .DECOUPLE_W ( 0              )
    ) i_axi2reg (
        .clk_i,
        .rst_ni,
        .testmode_i ( test_mode_i ),
        .in         ( axi_slave  ),
        .reg_o      ( axi_to_regfile )
    );

    // REG_BUS interface to struct conversion
    typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
    typedef logic [AXI_DATA_WIDTH-1:0] data_t;
    typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;

    `REG_BUS_TYPEDEF_REQ(reg_req_t, addr_t, data_t, strb_t)
    `REG_BUS_TYPEDEF_RSP(reg_rsp_t, data_t)

    reg_req_t to_reg_file_req;
    reg_rsp_t from_reg_file_rsp;

    `REG_BUS_ASSIGN_TO_REQ(to_reg_file_req, axi_to_regfile)
    `REG_BUS_ASSIGN_FROM_RSP(axi_to_regfile, from_reg_file_rsp)

    // Autogenerated register file
    wide_alu_reg_top #(
        .reg_req_t(reg_req_t),
        .reg_rsp_t(reg_rsp_t)
    ) i_regfile (
        .clk_i,
        .rst_ni,
        .devmode_i(1'b1),
        .reg_req_i(to_reg_file_req),
        .reg_rsp_o(from_reg_file_rsp),
        .reg2hw(reg_file_to_ip),
        .hw2reg(ip_to_reg_file)
    );

endmodule
```

Notes

- `devmode_i` is tied to `1'b1` as recommended for standalone IPs
- `reg2hw` and `hw2reg` are **struct-level connections**, not individual signals
- The register file is now fully reachable from AXI, but still not connected to hardware

Step Summary

In this step, we instantiated the reggen-generated register file and connected it to the AXI-to-register protocol converter using struct-based request and response signals. The register file now exposes typed hardware-facing interfaces (`reg2hw` and `hw2reg`) that will be connected to the wide ALU accelerator in the next step.

Step 3.9 – Instantiating and Wiring the `wide_alu` Accelerator

In this step, we instantiate the actual **`wide_alu` hardware accelerator** and connect it to the autogenerated register file using the reggen-generated wiring structs.

This completes the wrapper by:

- Driving hardware inputs from register fields
- Returning results and status back into registers
- Correctly handling write-enable (`qe`) semantics for control signals

After this step:

- The AXI interface can fully control the accelerator
- Software-visible registers are functionally connected to hardware
- The wrapper `wide_alu_top` is complete

File to Modify

`$HOME/GP/rtl/wide_alu/wide_alu_top.sv`

Code Changes in This Step

Add the following **at the end of the module**, after the register file instantiation:

```
    // Instance of the actual Accelerator
    wide_alu i_wide_alu (
        .clk_i,
        .rst_ni,

        // Control signals
        .trigger_i   ( reg_file_to_ip.ctrl1.trigger.q &
                       reg_file_to_ip.ctrl1.trigger.qe ),
        .clear_err_i ( reg_file_to_ip.ctrl1.clear_err.q &
                       reg_file_to_ip.ctrl1.clear_err.qe ),

        // Operands
        .op_a_i ( reg_file_to_ip.op_a ),
        .op_b_i ( reg_file_to_ip.op_b ),

        // Delay / deacceleration control
        .deaccel_factor_we_i ( reg_file_to_ip.ctrl2.delay.qe ),
        .deaccel_factor_i    ( reg_file_to_ip.ctrl2.delay.q ),
        .deaccel_factor_o    ( ip_to_reg_file.ctrl2.delay.d ),
```

```
        // Operation select
        .op_sel_we_i ( reg_file_to_ip.ctrl2.opsel.qe ),
        .op_sel_i    ( wide_alu_pkg::optype_e'
                       ( reg_file_to_ip.ctrl2.opsel.q ) ),
        .op_sel_o    ( ip_to_reg_file.ctrl2.opsel.d ),

        // Outputs
        .result_o ( ip_to_reg_file.result ),
        .status_o ( ip_to_reg_file.status.d )
    );
```

Current State of `wide_alu_top.sv` (FULL Wrapper)

```
`include "register_interface/typedef.svh"

`include "register_interface/assign.svh"

//The wrapper instantiates the custom IP ,connects it with the auto generated
register file and it converts the generic register interface protocol to
AXI // protocol so we can expose just a single AXI slave port at the top
level

module wide_alu_top #(

    parameter int unsigned AXI_ADDR_WIDTH = 32,

    localparam int unsigned AXI_DATA_WIDTH = 32,

    parameter int unsigned AXI_ID_WIDTH = -1,

    parameter int unsigned AXI_USER_WIDTH = -1

) (

    input  logic    clk_i,

    input  logic    rst_ni,

    input  logic    test_mode_i,


    AXI_BUS.Slave   axi_slave // To be converted with the protocol
converter(3) to register_interface bus

);

// Importing the auto generated system verilog package

    import wide_alu_reg_pkg::wide_alu_reg2hw_t;

    import wide_alu_reg_pkg::wide_alu_hw2reg_t;

    // 1. Declare the internal Register Bus interface

    REG_BUS #(.ADDR_WIDTH(32), .DATA_WIDTH(32)) axi_to_regfile();

    // 2. Wiring signals for the generated register structs
```

```
    wide_alu_reg2hw_t reg_file_to_ip; // Also structs one from the direction
to the ip

    wide_alu_hw2reg_t ip_to_reg_file; // From ip to the register file

    // 3. Instantiate the AXI to Register Interface converter

    axi_to_reg_intf #(

        .ADDR_WIDTH(AXI_ADDR_WIDTH),

        .DATA_WIDTH(AXI_DATA_WIDTH),

        .ID_WIDTH(AXI_ID_WIDTH),

        .USER_WIDTH(AXI_USER_WIDTH),

        .DECOUPLE_W(0)

    ) i_axi2reg (

        .clk_i,

        .rst_ni,

        .testmode_i(test_mode_i),

        .in(axi_slave),

        .reg_o(axi_to_regfile)

    );

    // 4. Convert the REG_BUS interface to the struct signals used by
autogenerated register file (macro magic to convert struct)

    typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;

    typedef logic [AXI_DATA_WIDTH-1:0] data_t;

    typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;

// Declaration of structs

    `REG_BUS_TYPEDEF_REQ(reg_req_t, addr_t, data_t, strb_t)

    `REG_BUS_TYPEDEF_RSP(reg_rsp_t, data_t)

//Wiring of structs

    reg_req_t to_reg_file_req; // ONE FROM THE AXI TO THE REGISTER FILE
PROTOCOL CONVERTER

    reg_rsp_t from_reg_file_rsp; // ONE FROM THE THE REGISTER FILE TO
PROTOCOL CONVERTER

// AACtual conversion between system verilog interface and the structs

    `REG_BUS_ASSIGN_TO_REQ(to_reg_file_req, axi_to_regfile)
```

```
    `REG_BUS_ASSIGN_FROM_RSP(axi_to_regfile, from_reg_file_rsp)

    // 5. Instance of the auto-generated Register File

    wide_alu_reg_top #(

        .reg_req_t(reg_req_t), // Type parameters in the auto generated
wide_alu_top_reg.sv

        .reg_rsp_t(reg_rsp_t)

    ) i_regfile (

        .clk_i,

        .rst_ni,

        .devmode_i(1'b1),

        // From the protocol converter

        .reg_req_i(to_reg_file_req),//wiring the signals of the structs

        .reg_rsp_o(from_reg_file_rsp),

        // Signals to wide_alu IP

        .reg2hw(reg_file_to_ip),

        .hw2reg(ip_to_reg_file)

    );

    // 6. Instance of the actual Accelerator wide alu

    wide_alu i_wide_alu (

        .clk_i,

        .rst_ni,

        .trigger_i(reg_file_to_ip.ctrl1.trigger.q &
reg_file_to_ip.ctrl1.trigger.qe),

        .clear_err_i(reg_file_to_ip.ctrl1.clear_err.q &
reg_file_to_ip.ctrl1.clear_err.qe),

        .op_a_i(reg_file_to_ip.op_a),

        .op_b_i(reg_file_to_ip.op_b),

        .deaccel_factor_we_i(reg_file_to_ip.ctrl2.delay.qe),

        .deaccel_factor_i(reg_file_to_ip.ctrl2.delay.q),

        .deaccel_factor_o(ip_to_reg_file.ctrl2.delay.d),

        .op_sel_we_i(reg_file_to_ip.ctrl2.opsel.qe),

        .op_sel_i(wide_alu_pkg::optype_e'(reg_file_to_ip.ctrl2.opsel.q)),
```

```
        .op_sel_o(ip_to_reg_file.ctrl2.opsel.d),
        .result_o(ip_to_reg_file.result),
        .status_o(ip_to_reg_file.status.d)
    );
endmodule
```

Notes

- Signals using `.q & .qe`:
  - Represent **edge-triggered or write-triggered controls**
  - Only assert when software writes the register
- Signals using only `.q`:
  - Represent stable configuration values
- Signals using `.d`:
  - Are written back into registers by hardware
- `hwext: true` fields in the HJSON explain why `.qe` is required for some signals

Step Summary

In this step, we instantiated the `wide_alu` accelerator and fully wired it to the autogenerated register file. Control, configuration, operands, results, and status are now correctly exchanged between software-accessible registers and hardware logic. This completes the `wide_alu_top` wrapper and makes the accelerator AXI-accessible.

Step 4.1 – Instantiating `wide_alu_top` in `pulp_soc.sv`

At this stage, our objective is **only** to instantiate the `wide_alu_top` wrapper inside `pulp_soc.sv` and prepare a dedicated AXI slave bus for it.The AXI bus will be declared and wired locally in `pulp_soc`, but it will not be connected to the crossbar until the next step.

In `pulp_soc.sv`

1. Declare a dedicated AXI slave bus for the Wide ALU

In `pulp_soc.sv`, declare a new AXI bus signal that will later be connected to the SoC interconnect:

```
AXI_BUS #(
        .AXI_ADDR_WIDTH ( AXI_ADDR_WIDTH      ),
        .AXI_DATA_WIDTH ( AXI_DATA_OUT_WIDTH  ),
        .AXI_ID_WIDTH   ( AXI_ID_OUT_WIDTH    ),
```

```
        .AXI_USER_WIDTH ( AXI_USER_WIDTH      )

    ) s_wide_alu_bus();
```

Why this is needed

- This bus represents the **AXI slave interface** of the Wide ALU
- For now, it exists only locally in `pulp_soc`
- In the next step, this bus will be routed into `soc_interconnect_wrap`

2. Instantiate the `wide_alu_top` wrapper

Instantiate the wrapper module:

```
wide_alu_top #(
  .AXI_ADDR_WIDTH ( AXI_ADDR_WIDTH     ),
  .AXI_ID_WIDTH   ( AXI_ID_OUT_WIDTH   ),
  .AXI_USER_WIDTH ( AXI_USER_WIDTH     )
) i_wide_alu (
  .clk_i        ( s_soc_clk       ),
  .rst_ni       ( s_soc_rstn      ),
  .test_mode_i  ( dft_test_mode_i ),
  .axi_slave    ( s_wide_alu_bus  )
);
```

Notes

- `clk_i` / `rst_ni`
  → Connected to the SoC clock and reset domain, same as other peripherals
- `test_mode_i`
  → Passed through from the SoC DFT test mode
- `axi_slave`
  → Connected to the newly declared `s_wide_alu_bus`

At this point, the Wide ALU is **instantiated but isolated**.

Step summary

- `wide_alu_top` is instantiated in `pulp_soc.sv`
- A dedicated AXI slave bus (`s_wide_alu_bus`) exists
- Clock, reset, and test-mode wiring is correct
- The AXI bus is not yet connected to the SoC interconnect

The Wide ALU is now **present in the SoC hierarchy**, ready to be exposed to the AXI crossbar in the next step.]

Step 5.1 – Extending `soc_interconnect_wrap` with a New AXI Slave Port

Expose the **Wide ALU accelerator** to the SoC AXI interconnect by adding a **third AXI slave port**.At this stage, we only extend the **structure and wiring** of the AXI crossbar.

File to Modify

~/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc/
`soc_interconnect_wrap.sv`

Modification 1 – Increase the Number of AXI Slave Ports

The AXI crossbar was originally instantiated with **2 AXI slave ports**.
Since the Wide ALU is an additional AXI-mapped peripheral, the number of AXI slave ports must be increased to **3**.

This affects:

- The AXI interconnect parameters
- The size of the AXI slave interface array

```
.NR_AXI_SLAVE_PORTS(3),
```

Change:

- AXI slave count: **2 → 3**

Modification 2 – Add a New AXI Port to the Module Interface

In the port list of `soc_interconnect_wrap`, add a new AXI port for the Wide ALU.

Add port:

```
AXI_BUS.Master wide_alu_slave
```

Modification 3 – Resize the AXI Slave Interface Array

Inside `soc_interconnect_wrap instantiation`, resize the AXI slave interface (AXI_BUS) array to hold the new port.

Before:

```
axi_slaves[2]();
```

After:

```
axi_slaves[3]();
```

This allocates a third AXI slot specifically for the Wide ALU.

Modification 4 – Connect the Wide ALU to the AXI Slave Array

Connect the new AXI port to the third entry of the AXI slave array.

Connection mapping:

- `axi_slaves[0]` → existing AXI slave (AXI plug)
- `axi_slaves[1]` → existing AXI slave (AXI-lite bridge)
- `axi_slaves[2]` → **Wide ALU**

Assignment:

```
`AXI_ASSIGN(wide_alu_slave, axi_slaves[2])
```

This makes the Wide ALU visible to the AXI crossbar infrastructure.

Step Summary

- Increased AXI slave port count from **2 to 3**
- Added a new AXI port named `wide_alu_slave`
- Resized the `axi_slaves` interface array
- Connected the Wide ALU to `axi_slaves[2]`
- **No address decoding added yet**

Modification 5 – Connect the New AXI Port to the dedicated AXI slave

Look at the last added line

.wide_alu_slave (s_wide_alu_bus)

```
soc_interconnect_wrap #(
  .NR_HWPE_PORTS(NB_HWPE_PORTS),
  .NR_L2_PORTS(NB_L2_BANKS),
  .AXI_IN_ID_WIDTH(AXI_ID_IN_WIDTH),
  .AXI_USER_WIDTH(AXI_USER_WIDTH)
) i_soc_interconnect_wrap (
    .clk_i               ( s_soc_clk          ),
    .rst_ni              ( s_soc_rstn         ),
    .test_en_i           ( dft_test_mode_i    ),
    .tcdm_fc_data        ( s_lint_fc_data_bus  ),
    .tcdm_fc_instr       ( s_lint_fc_instr_bus ),
    .tcdm_udma_rx        ( s_lint_udma_rx_bus  ),
    .tcdm_udma_tx        ( s_lint_udma_tx_bus  ),
    .tcdm_debug          ( s_lint_debug_bus    ),
    .tcdm_hwpe           ( s_lint_hwpe_bus     ),
    .axi_master_plug     ( s_data_in_bus       ),
    .axi_slave_plug      ( s_data_out_bus      ),
    .apb_peripheral_bus  ( s_apb_periph_bus    ),
    .l2_interleaved_slaves ( s_mem_l2_bus      ),
    .l2_private_slaves   ( s_mem_l2_pri_bus    ),
    .boot_rom_slave      ( s_mem_rom_bus       ),
    .wide_alu_slave      ( s_wide_alu_bus      )
    );
```

Step 5.2 – Add AXI Address Decoding Rules for the Wide ALU

At this point, the Wide ALU is physically connected to the AXI crossbar as an additional AXI slave interface.What is still missing is **address decoding**: the crossbar must know **which address range maps to which AXI slave index**.

This step assigns a **memory-mapped address range** to the Wide ALU and connects that range to the newly added AXI slave slot.

Files Modified

- `rtl/includes/soc_mem_map.svh`
- ~/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc/`soc_interconnect_wrap.sv`

Step 5.2.1 – Define the Wide ALU memory-mapped address range

File: `rtl/includes/soc_mem_map.svh`

Add a new address window for the Wide ALU registers.
This range must be:

- AXI-accessible
- Non-overlapping with existing regions
- Large enough to cover the reggen-generated register file

```
`define SOC_MEM_MAP_WIDE_ALU_START_ADDR 32'h1A40_0000
`define SOC_MEM_MAP_WIDE_ALU_END_ADDR   32'h1A40_1000
```

Notes

- Size: `0x1000` (4 KB), more than enough for the register file
- Address chosen to sit after the peripheral region
- These macros will be referenced by the AXI crossbar rules

Step 5.2.2 – Extend the AXI crossbar rule table

File: ~/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc/`soc_interconnect_wrap.sv`

The AXI crossbar uses a **rule table** that maps:

```
[address range] → [AXI slave index]
```

Since we added a **third AXI slave**, we must:

1. Increase the number of rules
2. Add one rule that targets the new slave index

Increase the number of AXI rules

```
localparam int unsigned NR_RULES_AXI_CROSSBAR = 3;
```

Step 5.2.3 – Add the Wide ALU address rule

Extend the rule table with a new entry that maps the Wide ALU address range to **AXI slave index 2**:

```
localparam addr_map_rule_t [NR_RULES_AXI_CROSSBAR-1:0] AXI_CROSSBAR_RULES =
'{
  '{ idx: 0, start_addr: `SOC_MEM_MAP_AXI_PLUG_START_ADDR,
             end_addr:   `SOC_MEM_MAP_AXI_PLUG_END_ADDR },

  '{ idx: 1, start_addr: `SOC_MEM_MAP_PERIPHERALS_START_ADDR,
             end_addr:   `SOC_MEM_MAP_PERIPHERALS_END_ADDR },

  '{ idx: 2, start_addr: `SOC_MEM_MAP_WIDE_ALU_START_ADDR,
             end_addr:   `SOC_MEM_MAP_WIDE_ALU_END_ADDR }
};
```

Key points

- `idx: 2` corresponds exactly to:
- `axi_slaves[2]`
- No new ports were created — the existing AXI slave array was extended
- The crossbar now correctly routes accesses in `0x1A40_0000–0x1A40_0FFF` to the Wide ALU

Step 6 download wide_alu , wide_alu_pkg files

Please, download these two files from this link wide_alu.sv , wide_alu_pkg.sv

https://drive.google.com/drive/folders/1iVXoUbCY0rCGpfaMywgJEAmJNe6lE7FX

and open them and take a look as the designer should write them like FFT.sv , FFT_pkg.sv

Step 7 add the wide_alu to the bender dependency

1. Edit your Root Bender.yml Open ~/GP/Bender.yml and add the wide alu to the source section. It should look like this: (case in the setup)

Sources:

……

- rtl/wide_alu/wide_alu_pkg.sv

- rtl/wide_alu/wide_alu.sv

- rtl/wide_alu/wide_alu_top.sv

- rtl/wide_alu/wide_alu_reg_pkg.sv

- rtl/wide_alu/wide_alu_reg_top.sv

2. Edit your Root Bender.yml Open ~/GP/Bender.yml and add register_interface to the dependencies section. It should look like this: (special case in the setup)

dependencies:

……

Add this line below:

register_interface: { git: "https://github.com/pulp-platform/register_interface.git", version: 0.3.1 }

Note : this is the special case in the setup file, where we need to import or include package, then you have to include it in the dependencies of the ~/GP/Bender.yml because here is the only case where the order of compilation is important

Since you add a new dependency you have to run this block inside ~/GP

./bender update

./bender script vsim >compile.tcl

./fix_bugs.sh

cd sim

make build

vopt +acc=npr -o vopt_tb tb_pulp -floatparameters+tb_pulp -work work

cd examples/hello

make clean all run

Step 8 Build and verify IP visibility to Bender

You should run any test like hello test and make sure it works and if there are some bugs you need to solve them before the software steps

Software Integration Steps:

Step 1 – Create the Driver Header and Implementation Files

In this step, we establish the software foundation within the `pulp-runtime`. We create the necessary files to house the driver's API definitions and the underlying logic required to communicate with the hardware accelerator.

Files to Create

IN: `pulp-runtime`

File path 1:

`/pulp-runtime/include/wide_alu_driver.h`

File path 2:

`/pulp-runtime/drivers/wide_alu_driver.c`

Modification

Create the empty files in their respective directories within the `pulp-runtime` workspace.

Commands:

From the root of the `pulp-runtime` directory:
Bash

```
touch include/wide_alu_driver.h
touch drivers/wide_alu_driver.c
```

Notes

- **include/wide_alu_driver.h**: This header file will contain the function prototypes and external definitions. It acts as the interface for the `main.c` application.
- **drivers/wide_alu_driver.c**: This implementation file will contain the C code that performs memory-mapped I/O (MMIO) to control the accelerator.
- The `drivers/` directory may need to be created if it does not already exist in your specific runtime version.

Explanation

- `pulp-runtime` organizes software by separating interface definitions (`include/`) from functional logic (`drivers/` or `lib/`).
- By placing these files in the standard runtime structure:
    - The compiler can easily locate the header during the build process of the test application.
    - The build system (`pulp.mk`) can be configured to compile the driver logic into the final binary.
- This step only creates the physical files; no code is added, and the build system is not yet aware of them.

Step Summary

- Created the skeleton `wide_alu_driver.h` in the include directory.
- Created the skeleton `wide_alu_driver.c` in the drivers directory.
- Established the file-system foundation for the software driver integration.

Step 2 – Adding Driver Implementation and Header Content

In this step, the driver's logical structure is defined. The header file establishes the API while the implementation file contains the low-level functions required to write operands, configure operations, and poll for completion.

Files to Modify

IN: `pulp-runtime`

File path 1: `/pulp-runtime/include/wide_alu_driver.h`

File path 2: `/pulp-runtime/drivers/wide_alu_driver.c`

Modification

Populate the newly created files with the function declarations and their corresponding definitions.

`wide_alu_driver.h` Content:

```
#include <stdint.h>

#define WIDE_ALU_BASE_ADDR 0x1a400000

void set_op(uint8_t operation);

void set_delay(uint8_t delay);

void trigger_op(void);

void set_operands(uint32_t* a, uint32_t* b);

int poll_done(void);

void clear_error(void);

int wide_multiply(uint32_t a[32], uint32_t b[32], uint32_t result[64]);
```

`wide_alu_driver.c` Content as per repo:

```
#include <wide_alu_driver.h>

#include <stdint.h>

#include <stdio.h>

void set_delay(uint8_t delay)

{

  uint32_t volatile * ctrl2_reg = (uint32_t*)WIDE_ALU_CTRL2(0);

  uint32_t ctrl2_old_value;

  //Read old value

  ctrl2_old_value = *ctrl2_reg;

  //Overwrite operation bits

  *ctrl2_reg = ctrl2_old_value | (delay &
WIDE_ALU_CTRL2_DELAY_MASK)<<WIDE_ALU_CTRL2_DELAY_LSB;

}

void set_operands(uint32_t* a, uint32_t* b)

{
```

```
  uint32_t volatile * op_a_reg_start = (uint32_t*)WIDE_ALU_OP_A_0(0);
  uint32_t volatile * op_b_reg_start = (uint32_t*)WIDE_ALU_OP_B_0(0);
  //Make sure we are in idle state before changing the operands
  for (int i = 0; i<256/32; i++)
  {
    op_a_reg_start[i] = a[i];
    op_b_reg_start[i] = b[i];
  }
}
int poll_done(void)
{
  uint32_t volatile * status_reg = (uint32_t*)WIDE_ALU_STATUS(0);
  uint32_t current_status;
  do {
    current_status =
(*status_reg>>WIDE_ALU_STATUS_CODE_LSB)&WIDE_ALU_STATUS_CODE_MASK;
  } while(current_status == WIDE_ALU_STATUS_CODE_PENDING);
  if (current_status == WIDE_ALU_STATUS_CODE_IDLE)
    return 0;
  else
    return current_status;
}
void get_result(uint32_t* result)
{
  uint32_t volatile * result_reg_start = (uint32_t*)WIDE_ALU_RESULT_0(0);
  for (int i = 0; i<512/8; i++)
  {
    result[i] = result_reg_start[i];
  }
}
void clear_error(void)
{
  uint32_t volatile * ctrl1_reg = (uint32_t*)WIDE_ALU_CTRL1(0);
  //Trigger operation by writing to trigger bit
```

```
  *ctrl1_reg = (1 &
WIDE_ALU_CTRL1_CLEAR_ERR_MASK)<<WIDE_ALU_CTRL1_CLEAR_ERR_LSB;
}
```

A Potential Problem

There's a small but critical **index bug** in the `get_result` loop that may cause a memory corruption (buffer overflow).

The Bug: Result Loop Bounds

In `get_result`:

```
for (int i = 0; i < 512/8; i++)
```

- **512 / 8 = 64**.
- However, your `RESULT` multireg in the hardware (and your `wide_alu_regs.h`) only has **16 registers** (`WIDE_ALU_RESULT_MULTIREG_COUNT 16`).
- If you loop 64 times, the core will try to read 48 addresses past the end of your IP, which will likely cause a **Bus Error** or read garbage.

The Final Step for this File

Update your `wide_alu_driver.c` with the fixed bounds and the missing `set_op`, `trigger_op`, and `wide_multiply` functions required by your instructions.

the corrected and completed implementation:

```
#include <wide_alu_driver.h>
#include <stdint.h>
#include <stdio.h>

// Helper: Set the number of cycles for the operation
void set_delay(uint8_t delay) {
    uint32_t volatile * ctrl2_reg = (uint32_t*)WIDE_ALU_CTRL2(0);
    uint32_t ctrl2_old_value = *ctrl2_reg;
    // Clear old delay bits (mask is 0xFF, offset is 16)
    ctrl2_old_value &= ~(WIDE_ALU_CTRL2_DELAY_MASK <<
WIDE_ALU_CTRL2_DELAY_OFFSET);
    *ctrl2_reg = ctrl2_old_value | (delay & WIDE_ALU_CTRL2_DELAY_MASK) <<
WIDE_ALU_CTRL2_DELAY_OFFSET;
}

// Implement the required set_op function
void set_op(uint8_t operation) {
    uint32_t volatile * ctrl2_reg = (uint32_t*)WIDE_ALU_CTRL2(0);
    uint32_t ctrl2_old_value = *ctrl2_reg;
    // Overwrite the OPSEL bits (bits 2:0)
    ctrl2_old_value &= ~(WIDE_ALU_CTRL2_OPSEL_MASK <<
WIDE_ALU_CTRL2_OPSEL_OFFSET);
```

```
    *ctrl2_reg = ctrl2_old_value | (operation & WIDE_ALU_CTRL2_OPSEL_MASK)
<< WIDE_ALU_CTRL2_OPSEL_OFFSET;
}

// Implement the required trigger_op function
void trigger_op(void) {
    uint32_t volatile * ctrl1_reg = (uint32_t*)WIDE_ALU_CTRL1(0);
    *ctrl1_reg = (1 << WIDE_ALU_CTRL1_TRIGGER_BIT);
}

void set_operands(uint32_t* a, uint32_t* b) {
    uint32_t volatile * op_a_reg_start = (uint32_t*)WIDE_ALU_OP_A_0(0);
    uint32_t volatile * op_b_reg_start = (uint32_t*)WIDE_ALU_OP_B_0(0);
    for (int i = 0; i < 8; i++) { // 256 bits / 32 bits = 8 regs
        op_a_reg_start[i] = a[i];
        op_b_reg_start[i] = b[i];
    }
}

int poll_done(void) {
    uint32_t volatile * status_reg = (uint32_t*)WIDE_ALU_STATUS(0);
    uint32_t current_status;
    do {
        current_status = (*status_reg >> WIDE_ALU_STATUS_CODE_OFFSET) &
WIDE_ALU_STATUS_CODE_MASK;
    } while(current_status == WIDE_ALU_STATUS_CODE_VALUE_PENDING);

    return (current_status == WIDE_ALU_STATUS_CODE_VALUE_IDLE) ? 0 :
current_status;
}

void get_result(uint32_t* result) {
    uint32_t volatile * result_reg_start =
(uint32_t*)WIDE_ALU_RESULT_0(0);
    for (int i = 0; i < 16; i++) { // FIXED: 512 bits / 32 bits = 16 regs
        result[i] = result_reg_start[i];
    }
}

// The main wrapper function required by your assignment
int wide_multiply(uint32_t a[8], uint32_t b[8], uint32_t result[16]) {
    set_operands(a, b);
    set_op(WIDE_ALU_CTRL2_OPSEL_VALUE_MUL);
    trigger_op();
    int status = poll_done();
    if (status == 0) {
        get_result(result);
    }
    return status;
}
```

Notes

- **Address Mapping:** The driver relies on macros like `WIDE_ALU_OP_A_0(0)` which must be defined in the autogenerated register header.
- **Operand Width:** The `set_operands` function handles two 256-bit inputs by iterating through eight 32-bit register writes.
- **Result Width:** The `get_result` function must be implemented to read back the 512-bit result (16 registers).

Explanation

- The **Header File** provides a contract for the application, abstracting the hardware details behind clear function names.
- The **Implementation File** performs the actual "work" by:
    - Converting 32-bit software arrays into the wide data types expected by the hardware.
    - Managing the control flow, such as ensuring the accelerator is in an `IDLE` state before a new operation begins.
- This step provides the logic but does not yet link it to the actual hardware base address or the build system.

Step Summary

- Populated `wide_alu_driver.h` with the API required to utilize the custom accelerator.
- Implemented the register-access logic in `wide_alu_driver.c`.
- Prepared the software stack to handle 256-bit operands and 512-bit results.

Step 3 – Generating the Register Map Header

In this step, we use the `regtool` utility to bridge the gap between the hardware description and the software driver. This tool automates the creation of a C header file that defines the exact memory offsets and bitfields required to control the Wide ALU.

Files Involved

Input File: `$HOME/GP/rtl/wide_alu/wide_alu_top.sv/wide_alu.hjson`

Output File: `$HOME/GP/pulp-runtime/include/wide_alu_regs.h`

Tool Location:
`/home/icpedia/pulp/ips/register_interface/vendor/lowrisc_opentitan/util/regtool.py`

Modification

Run the `regtool.py` (reggen) script with the C-defines flag to generate the register definition header.

Command Adjust to your paths:

```
python3 $REGGEN -D rtl/wide_alu/wide_alu.hjson > pulp-
runtime/include/wide_alu_regs.h
```

Notes

- **Tool Origin:** The `regtool.py` is sourced from the `register_interface` IP because it is specifically patched to support the Generic Register Interface Protocol used in this exercise.
- **-D Flag:** This flag instructs the tool to render C-style `#define` statements instead of SystemVerilog RTL or JSON.
- **Schema Compatibility:** The tool reads the `wide_alu.hjson` to determine register names (e.g., `CTRL1`, `OP_A`), access permissions (e.g., `wo`, `ro`), and bit widths.

Explanation

- The generated file, `wide_alu_regs.h`, acts as a "map" for the software driver.
- It provides symbolic constants for:
    - **Register Offsets:** Such as `WIDE_ALU_OP_A_0_REG_OFFSET` (0x0) and `WIDE_ALU_STATUS_REG_OFFSET` (0x88).
    - **Bitfield Masks:** Allowing the software to isolate specific bits like the `TRIGGER` bit or the `OPSEL` field.
- Using this generated header prevents manual errors that occur when hardcoding addresses and ensures that software is always synchronized with the hardware RTL.

Step Summary

- Located the compatible `regtool.py` utility within the PULP IP directory.
- Successfully generated `wide_alu_regs.h` containing all necessary C definitions.
- Provided the driver with a type-safe and symbolic way to access the Wide ALU peripheral registers.

Step 4 – Patching the Runtime Environment

This step integrates the newly created driver files into the `pulp-runtime` build system. By modifying the master header and the build rules, the Wide ALU driver becomes a permanent part of the SDK, allowing any application to access the accelerator's functionality.

Files to Modify

1. Master Header File

- **Repository:** `pulp-runtime`
- **File path:** `/pulp-runtime/include/pulp.h`

2. Build System Rules

- **Repository:** `pulp-runtime`
- **File path: /pulp-runtime/rules/pulpos/targets/`pulp.mk`**

Modification

1. Update `pulp.h` Add the following include directives to the end of the file (before the final `#endif`) to expose the driver API and register definitions to the user application:

```
#include "wide_alu_regs.h"
#include "wide_alu_driver.h"
```

2. Update `pulp.mk`

Locate the source file list (usually `PULP_SDK_SRCS`) and append the driver implementation file to ensure it is compiled during the build process:
Makefile

```
PULP_SRCS     += drivers/wide_alu_driver.c
```

Notes

- **Visibility:** Including `wide_alu_driver.h` in `pulp.h` ensures that the user only needs to include `#include <pulp.h>` in their `main.c` to access the accelerator.
- **Compilation:** Adding the `.c` file to `pulp.mk` ensures the linker can resolve the function symbols for `set_op` and `wide_multiply`.
- **Order:** Ensure `wide_alu_regs.h` is included before or alongside the driver header, as the driver logic depends on the generated macros.

Explanation

- `pulp.h` acts as the primary entry point for the PULPissimo software stack; patching it makes the custom IP a "first-class" peripheral.
- The `pulp.mk` file governs the compilation of the entire runtime library; without this modification, the functions defined in the driver would result in "undefined reference" errors during the linking stage of the test application.
- This step finalizes the environment setup, moving the project from the "configuration" phase to the "implementation" phase.

Step Summary

- Exposed the Wide ALU API and register map in the master SDK header (`pulp.h`).
- Registered the driver source file in the global build rules (`pulp.mk`).
- Successfully linked the software stack to the hardware-specific register definitions.

Step 5 – Initializing the Driver Source Code

In this step, we begin the actual coding phase by initializing the implementation file with its required dependencies. This ensures that the driver has access to the PULP system environment, the register map, and the correct hardware base address.

File to Modify

Repository: `pulp-runtime`

File path: `/pulp-runtime/drivers/wide_alu_driver.c`

Modification

Add the core include directives and the hardware base address definition to the top of the file.

Code to Add:
C

```c
#include "pulp.h"
#include "wide_alu_driver.h"
#include "wide_alu_regs.h"

// Define the base address for the hardware
#define WIDE_ALU_BASE 0x1A400000
```

Notes

- **`pulp.h`**: Provides access to basic SoC functions and hardware abstraction layers.
- **`wide_alu_driver.h`**: Ensures the implementation matches the function prototypes defined for the application.
- **`wide_alu_regs.h`**: Provides the symbolic names for register offsets generated before
- **`WIDE_ALU_BASE`**: This constant map the driver to the specific memory region (`0x1A40_0000`) where the accelerator was integrated into the SoC AXI crossbar.

Explanation

- These lines act as the "preamble" for the driver.
- Without including `wide_alu_regs.h`, the driver would not recognize register names like `CTRL1` or `OP_A`.
- Defining the `WIDE_ALU_BASE` here creates a single point of truth for the IP's location in the memory map, making the code easier to maintain if the hardware address changes.

Step Summary

- Initialized the implementation file with necessary system and local headers.

- Defined the memory-mapped base address for the Wide ALU IP.
- Prepared the source file for the implementation of arithmetic and control functions.

Step 6 – Defining the Peripheral Base Address in the SoC Map

In this step, we ensure the hardware base address is recognized by the SoC's architectural definitions. While the driver uses this address locally, adding it to the chip's central memory map ensures consistency across the entire `pulp-runtime` and prevents address conflicts.

Files to Modify

Repository: `pulp-runtime`

File path: `/pulp-runtime/include/archi/chips/pulp/memory_map.h`

Modification

Add the Wide ALU base address to the SoC memory map header. This file defines where all peripherals (like UART, GPIO, and now your Accelerator) live in the 32-bit address space.

Code to Add:

C

```c
/* new axi ip */
#define WIDE_ALU0_BASE_ADDR 0x1a400000
```

Notes

- **Address Consistency:** The address `0x1a400000` must match the hardware address decoding rules defined during the AXI crossbar integration.
- **`archi` Directory:** Files under `archi/chips/pulp/` are low-level architectural headers that describe the specific "flavor" of the PULPissimo chip you are simulating.
- **Prefixing:** Using a prefix like `WIDE_ALU0_` follows the PULP naming convention for peripherals.

Explanation

- **Centralization:** By adding the address to `memory_map.h`, you allow other parts of the system (like the kernel or other drivers) to know where the Wide ALU is located without re-defining it in every file.
- **Conflict Prevention:** Seeing all addresses in one file helps you verify that `0x1a400000` does not overlap with existing regions like L2 memory or the APB peripherals.

- **Driver Integration:** Now, instead of hardcoding `0x1a400000` inside your `.c` file, you can simply use the constant `WIDE_ALU0_BASE_ADDR`.

Step Summary

- Updated the architectural memory map header `memory_map.h`.
- Defined `WIDE_ALU0_BASE_ADDR` as `0x1a400000`.
- Synchronized the software's view of the SoC address space with the hardware's AXI decoding logic.

Step 7 – Validating the Stack with the Test Application

In this final software step, we create the test application (`main.c`) to validate the entire hardware/software stack. This application utilizes the driver API to communicate with the integrated AXI peripheral and verify that the system correctly computes the expected result.

File to Create

First run these commands

cd ~/GP/examples

mkdir axi_ip

touch main.c

File path: ~/GP/examples/axi_ip/main.c

Modification

Create the `main.c` file and populate it with the test logic to multiply two values using the hardware accelerator.

Code Content:

```c
#include <stdio.h>
#include "pulp.h"

int main()
{
    printf("Hello World!\n");

    uint32_t a[8];      // 256 bits total [cite: 20]
    uint32_t b[8];      // 256 bits total [cite: 20]
    uint32_t result[16]; // 512 bits total [cite: 20]

    // Initialize memory
```

```
    memset(a, 0, sizeof(a));
    memset(b, 0, sizeof(b));
    memset(result, 0, sizeof(result));

    // Set test operands
    a[0] = 3;
    b[0] = 5;

    // Configure hardware delay factor
    set_delay(50);

    // Execute hardware-accelerated multiplication [cite: 49, 52]
    wide_multiply(a, b, result);

    // Output result to UART
    printf("Result [0]: %d\n", result[0]);

    return 0;
}
```

Notes

- **Operand Sizing**: While the original code used larger arrays, the `wide_alu` hardware specifically expects **8 words (256 bits)** for operands and provides **16 words (512 bits)** for results.
- **Delay Parameter**: The `set_delay(50)` call tests the "deacceleration factor" logic within the IP wrapper.
- **Standard Output**: Successful execution will result in the string `Result [0]: 15` appearing on your terminal.

Explanation

- **`pulp.h` Inclusion**: This header provides the prototypes for your driver functions because of the patch applied in **Step 5.4**.
- **MMIO Orchestration**: The call to `wide_multiply` triggers a chain of events:
    1. Software writes to the AXI Slave.
    2. The protocol converter updates the Register File.
    3. The Hardware Accelerator processes the data.
- **Verification**: This test confirms that AXI address decoding, register-file wiring, and software driver logic are all correctly synchronized.

Step Summary

- Created the `main.c` validation application.
- Configured test operands (3 and 5) and a deacceleration factor.
- Verified the end-to-end functionality of the custom AXI IP integration.

Step 8 – Configuring the Application Makefile

In this final step, we configure the local `Makefile` for the test application. This file tells the PULP SDK which source files to compile for the Fabric Controller (FC) and which optimization levels to use.

File to Modify

File to Create

First run these commands

touch `Makefile`

File path: ~/GP/examples/axi_ip/ `Makefile`

Modification

Define the application name and source files, and include the standard PULP runtime rules.

Code Content:
Makefile

```
PULP_APP = main
PULP_APP_FC_SRCS = main.c
PULP_APP_HOST_SRCS = main.c
PULP_CFLAGS = -O3 -g

include $(PULP_SDK_HOME)/install/rules/pulp_rt.mk
```

Notes

- **`pulp_rt.mk`**: **No, you do not modify this file.** It is a "read-only" system file provided by the SDK that contains the standard compilation and simulation recipes (like `make clean all run`).
- **`PULP_APP_FC_SRCS`**: This variable specifies that `main.c` should be compiled for the Fabric Controller, which is the main RISC-V core in PULPissimo.
- **`PULP_CFLAGS`**: The `-O3` flag enables high-level optimizations, while `-g` includes debug symbols for simulation.

Explanation

- **Inclusion vs. Modification**: By *including* `pulp_rt.mk` instead of *modifying* it, you keep your project portable. The system-wide changes you made earlier in `pulp.mk` (Step 5.4) will automatically be pulled in by this include statement.

- **SDK Integration**: This Makefile acts as the bridge between your custom code and the entire PULP toolchain (compiler, linker, and simulator).
- **Execution**: Once this file is saved, you can run the command `make clean all run` to trigger the compilation of both your `main.c` and the `wide_alu_driver.c` we integrated into the SDK.

Step Summary

- Created the application-level `Makefile`.
- Pointed the build system to `main.c`.
- Linked the application to the standard PULP runtime rules without modifying core SDK files.

Step 9 – Final fixes

1. Fix pulp-runtime/include/wide_alu_driver.h

2. Fix pulp-runtime/drivers/wide_alu_driver.c

This file needs to be updated to use the **Offset Macros** correctly (adding them to the Base Address).

Please download the both of them from the following drive link

wide_alu_driver.h , wide_alu_driver.c

https://drive.google.com/drive/folders/1iVXoUbCY0rCGpfaMywgJEAmJNe6lE7FX

Step 10 – Final result

Finally you should got this result after running the test

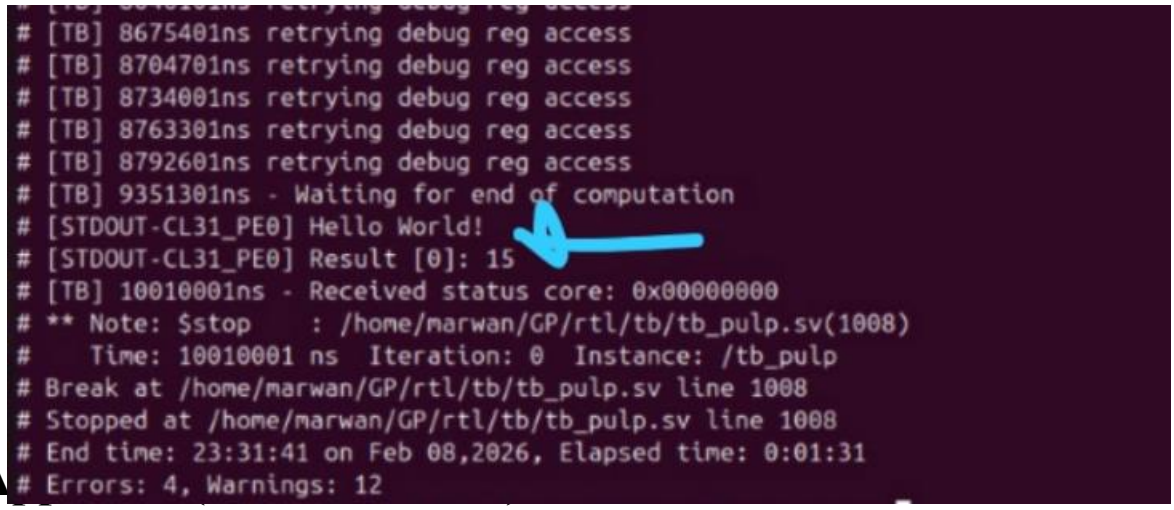
cd sim

make build

vopt +acc=npr -o vopt_tb tb_pulp -floatparameters+tb_pulp -work work

cd examples/axi_ip

make clean all run



These two files (`wide_alu.sv` and `wide_alu_pkg.sv`) are handwritten reference RTL, NOT generated.
They are **provided by the training authors** as a *golden example accelerator*.

- **Not generated by reggen**
- **Not autogenerated**
- **Intentionally written by a human**
- **Meant to be reused as-is in this exercise**

What is generated vs handwritten

1 Generated by reggen (tool output)

These come from `wide_alu.hjson`:

- `wide_alu_reg_pkg.sv`
- `wide_alu_reg_top.sv`

These files:

- Define register structs (`*_reg2hw_t`, `*_hw2reg_t`)
- Implement the register file logic
- Handle SW/HW access rules (`hwext`, `hwqe`, etc.)

You **never write these by hand**.

2 Handwritten accelerator RTL

These are **NOT** generated:

wide_alu_pkg.sv

```
package wide_alu_pkg;
  typedef enum logic[2:0] { ADD, SUB, MUL, XOR, AND, OR } optype_e;
  typedef enum logic[1:0] { IDLE, PENDING, ERROR_WRITE, ERROR_OPCODE }
status_e;
endpackage
```

Purpose:

- Defines **semantic meaning** of operations and states
- Used by:
  - `wide_alu.sv`
  - `wide_alu_top.sv`
  - regfile wiring (casts, enums)

This is **domain logic**, not infrastructure → must be handwritten.

wide_alu.sv

This is the **actual accelerator**.

It:

- Implements the 256-bit datapath
- Handles:
  - Trigger semantics
  - Deacceleration delay
  - Error handling
  - Status reporting
- Matches the **exact register semantics** defined in HJSON

This is **the IP you are integrating**, not something reggen can infer.

Why reggen can't generate this

Reggen only knows:

- Registers
- Fields
- Access permissions

It **does NOT know**:

- What ADD/MUL/XOR actually mean
- How to pipeline operations
- How long an operation takes

- How status transitions work

That logic **must be written manually**.

Nothing is conceptually missing.

Key takeaway (this is gold)

Reggen generates the "register plumbing", not the IP.
The IP itself is always handwritten.

Conceptual Explanation

You are connecting **three different worlds**:

1. **AXI** (used by the core / SoC)
2. **REG_BUS interface** (used by the AXI→reg protocol converter)
3. **reggen-generated register file**, which **does NOT use interfaces**, but **plain structs**

The problem is:

- `axi_to_reg_intf` outputs a **SystemVerilog interface** (`REG_BUS`)
- `wide_alu_reg_top` expects **structs** (`reg_req_t`, `reg_rsp_t`)

So we need: a **lossless adapter** between an *interface* and *structs*

That is exactly what these macros do.

What each part does

1. Struct type declarations

```
`REG_BUS_TYPEDEF_REQ(reg_req_t, addr_t, data_t, strb_t)
`REG_BUS_TYPEDEF_RSP(reg_rsp_t, data_t)
```

These macros:

- **define two packed structs**
- matching the Generic Register Interface protocol

They expand into something conceptually like:

```
typedef struct packed {
  addr_t addr;
  logic  write;
  data_t wdata;
```

```
  strb_t wstrb;
  logic  valid;
} reg_req_t;

typedef struct packed {
  data_t rdata;
  logic  error;
  logic  ready;
} reg_rsp_t;
```

So now we have **strongly-typed structs** for requests and responses.

2. Declaring the struct wires

```
reg_req_t to_reg_file_req;
reg_rsp_t from_reg_file_rsp;
```

These are the **actual wires** that will connect:

- AXI → regfile (`to_reg_file_req`)
- regfile → AXI (`from_reg_file_rsp`)

3. Converting interface ⇄ struct

```
`REG_BUS_ASSIGN_TO_REQ(to_reg_file_req, axi_to_regfile)
`REG_BUS_ASSIGN_FROM_RSP(axi_to_regfile, from_reg_file_rsp)
```

These macros:

- generate **continuous assignments**
- field-by-field
- between an **interface** and a **struct**

From the macro source you pasted, this expands to logic like:

```
assign to_reg_file_req = '{
  addr  : axi_to_regfile.addr,
  write : axi_to_regfile.write,
  wdata : axi_to_regfile.wdata,
  wstrb : axi_to_regfile.wstrb,
  valid : axi_to_regfile.valid
};

assign axi_to_regfile.rdata = from_reg_file_rsp.rdata;
assign axi_to_regfile.error = from_reg_file_rsp.error;
assign axi_to_regfile.ready = from_reg_file_rsp.ready;
```

So:

- **Interface → struct** for requests

- **Struct → interface** for responses

No logic, no latency, no behavior — just **signal mapping**.

Why this exists

- Many EDA tools struggle with interfaces inside autogenerated code
- reggen intentionally uses **plain structs**
- ETH's `register_interface` library provides **clean macros** to bridge both worlds
- This keeps autogenerated RTL **tool-agnostic and robust**

This is **not glue code** — it's a **design pattern**.